3-Phase Phase-Balanced Network Anomaly Detection System and Method Using Time-based Dynamic Keys

Abstract This paper proposes an active cyber defense technology that distributes network traffic across three virtual channels with a 120-degree phase difference and calculates real-time variable phases using time-based dynamic keys. Authorized packets maintain an equilibrium state where the sum of energy squares converges to a constant value of 1.5, while unauthorized access leads to detectable phase imbalance. The system quantifies these anomalies via the Equilibrium Deviation Index (EDI) to perform real-time intrusion detection. By utilizing DPDK and eBPF, the system ensures high-speed network performance and provides seamless connectivity for authorized users through automatic synchronization and sliding window correction.

---

1. Introduction

1.1 Technical Field

The present invention relates to a network security and intrusion detection system. Specifically, it pertains to an active cyber defense technology that distributes network traffic into three phases and detects abnormal access in real-time by ensuring that the sum of the phases, calculated through a time-based dynamic key shared only by authorized users, maintains a constant equilibrium.

1.2 Background of the Invention

Conventional network security relies on static IP allocation and firewall policies. Consequently, once a hacker successfully intrudes and seizes permissions, it is difficult to prevent them from staying in the network for a long time to leak data. Although using dynamic IPs can increase security, there remains a technical challenge where the connection continuity for legitimate users is compromised. The present invention models the network itself as a 3-phase balanced system oscillating at a specific rhythm, treating the hacker's presence as noise that disrupts the system's equilibrium.

---

2. Objectives and Solutions

2.1 Problem to Solve

The objective is to immediately detect a hacker's continuous occupation of the network as noise that breaks the equilibrium state and to fundamentally block reverse engineering by unauthorized persons by introducing a time-based dynamic phase key.

- **Phase-based Authentication:** Traffic phases are aligned via a time key shared between authorized users and the server to build a mathematical rhythm that a third party cannot replicate.
- **Real-time Anomaly Detection:** By detecting the phase drift that occurs when a hacker continuously connects or packets flow in, intrusions are confirmed as soon as the equilibrium (the promised total energy value) is broken.
- **Maintaining User Convenience:** Even if the IP changes dynamically, time-based phase synchronization ensures seamless communication for legitimate users.

2.2 Means for Solving the Problem

The system includes the following core components:

1. **3-Phase Packet Splitter (110):** Maps traffic to three channels with a 120-degree phase difference.
2. **Time-Synchronized Phase Generator (120):** Generates phase reference points based on a real-time varying time key (400).
3. **Equilibrium Monitoring Module (130):** Monitors whether the sum of the energy values (E) of each channel always maintains the promised total energy value (e.g., $sin^2\theta + sin^2(\theta + 2\pi/3) + sin^2(\theta + 4\pi/3) = 1.5$).
4. **Intrusion Decision and Response Unit (140):** Terminates sessions when the energy deviates from the threshold.

---

3. Detailed Description of the Invention

3.1 Temporal Synchronized Phase Modulation (TSPM)

In this system, time (T) is defined as a 'Dynamic Function Variable' that generates phase signals rather than a simple record. The authorized user terminal (200) and the security server share the same time-based seed value through a pre-defined algorithm.

- **Phase Generation Mechanism:** The system calculates a non-deterministic frequency constant using the current timestamp (T) as input. The instantaneous phase ϕ_n of each channel (u_n) is determined by: $\phi_n(t, T) = f(T) \cdot t + 2\pi(n-1)/3 + \alpha(T)$, where $f(T)$ is the variable frequency and $\alpha(T)$ is the dynamic phase offset.

- **Energy Aggregation and Integrity Verification:** Packets transmitted by authorized users maintain strictly aligned phase differences according to the formula. The total energy (E_{total}) calculated at the server receiver converges to a constant value of 1.5.

- **Security Barrier:** Even if an attacker (300) forges a packet, since they do not know the real-time values of $f(T)$ and $\alpha(T)$, the signals they transmit cause phase interference, leading the total energy to deviate from the 1.5 axis.

3.2 Packet Bit-stream Phase Mapping and Distributed Processing

The system goes beyond simply splitting IP packet data; it transforms the data into physical properties of phase (amplitude and phase offset).

- User data streams are collected in 24-bit units and then allocated to three channels ($u1, u2, u3$) as upper/middle/lower 8-bit octets.

- Each 8-bit data value (0-255) is mapped to the maximum amplitude (A) of each phase.

- For authorized users, the condition $A1 = A2 = A3$ (data balance) combined with the time-key-based phase difference (120 degrees) restores the total energy to 1.5 at the receiver. Hacker packets cause amplitude imbalances due to irregular bit arrangements, leading to an immediate rise in the Equilibrium Deviation Index (EDI).

3.3 DPDK-based Kernel-level Implementation

To perform real-time phase calculations for large volumes of packets, the system minimizes kernel interrupt latency.

- **DPDK (Data Plane Development Kit):** Processes packets directly in the user space to remove overhead from kernel mode switching, enabling 3-phase energy sum calculations even at Gbps bandwidth.

- **eBPF (Extended Berkeley Packet Filter):** Immediately inspects packets at the Network Interface Card (NIC) level and drops packets exceeding the imbalance threshold before they consume system resources.

3.4 Jitter and Clock Drift Correction

To maintain the integrity of the time-based key (400), fine time deviations between the server and client are managed.

- **PTP (Precision Time Protocol):** Uses the IEEE 1588 standard to maintain microsecond-level synchronization.
- **Sliding Window Verification:** To correct phase differences $\Delta\phi = f(T) \cdot d$ caused by network latency (d), the server generates multiple phase planes within the range $T \pm \Delta T$ to verify integrity and prevent false positives.

---

4. Anomaly Detection and Response

4.1 Intrusion Detection via Asymmetric Load Analysis

The network's equilibrium is monitored using the 'Balanced Load' concept from electrical engineering. Normal traffic distributes energy evenly across the three channels, while hacking generates an 'Asymmetric Load' concentrated on specific channels.

- **Brute-force/Scanning:** Large volumes of short-period packets cause high-frequency noise (harmonics) in the phase graph and jitter in the reference axis.
- **Persistence:** If a hacker occupies an IP for a long time, the amplitude of that specific phase continuously rises, exceeding the 'Phase Imbalance Index' threshold.

4.2 Equilibrium Deviation Index (EDI) Calculation

The system calculates the EDI at each sampling period: $EDI = \sqrt{(1/N) \cdot \sum(E_{total}(i) - 1.5)^2}$. If the EDI exceeds the set security level (e.g., 0.05), it is considered an immediate intrusion. Upon detection, the system disrupts the phase synchronization of the session or silently redirects the traffic to a virtual sandbox (400) to analyze the attack pattern.

---

5. Conclusion and Effects

5.1 Industrial Applicability

- **Critical Infrastructure:** Security for control networks requiring high stability (Smart grids, nuclear power plants).

- **Finance and Data Centers:** Ensures transaction integrity and prevents hacker residency via time-varying keys.
- **V2X (Autonomous Driving):** Detects external interference as phase imbalance to ensure driving safety.

5.2 Expected Effects

- **Active Security:** Fundamentally blocks reverse engineering through the TSPM mechanism.
- **Precision Detection:** Real-time detection of diverse hacking behaviors via EDI based on the 1.5 energy constant.
- **Real-time Performance:** Supports Gbps bandwidth without performance degradation using DPDK and eBPF.
- **User Convenience:** Provides seamless network environments through automatic synchronization and sliding window correction without complex password entries.

5.3 Test source code

simulator.html

```
<h1>3-Phase Phase-Balanced System Simulator</h1>
<p class="subtitle">Visualization of E_total = 1.5 Equilibrium Maintenance and EDI (Equil

<div class="dashboard">
    <div class="panel controls">
        <button id="modeToggle" class="toggle-btn">Authorized Mode</button>

        <hr style="border-color: #3a3a50; margin: 10px 0;">

        <div>
            <label>Base Frequency (Variable Time Key): <span id="freqVal">1.0</span>x</la
            <input type="range" id="frequency" min="0.5" max="3" step="0.1" value="1">
        </div>

        <div style="margin-top: 20px;">
            <h3 style="color:#e74c3c; margin-bottom: 10px;">Hacker Attack Simulation Sett
            <label>Asymmetric Load (Specific Phase Amplitude Increase): <span id="ampVal"
            <input type="range" id="amplitude" min="1" max="2.5" step="0.1" value="1" dis

            <br>
            <label>High-Frequency Noise (Scanning/Jitter): <span id="noiseVal">0.0</span>
            <input type="range" id="noise" min="0" max="0.5" step="0.05" value="0" disabl
```

```
        </div>
    </div>

    <div class="panel monitor">
        <div class="status-box">
            <div class="metric">
                <div style="color: #a0a0b0; font-size: 14px;">Real-time Total Energy (E_t
                <div id="totalEnergy" class="metric-value normal">1.500</div>
            </div>
            <div class="metric">
                <div style="color: #a0a0b0; font-size: 14px;">Equilibrium Deviation Index
                <div id="ediScore" class="metric-value normal">0.000</div>
            </div>
            <div class="metric">
                <div style="color: #a0a0b0; font-size: 14px;">System Status</div>
                <div id="systemStatus" class="metric-value normal">Safe</div>
            </div>
        </div>

        <canvas id="waveCanvas" width="600" height="300"></canvas>
        <div style="text-align: center; font-size: 12px; color: #777; margin-top: 10px;">
            <span style="color: #ff9ff3;">Phase 1</span> |
            <span style="color: #feca57;">Phase 2</span> |
            <span style="color: #48dbfb;">Phase 3</span> |
            <span style="color: #ff6b6b; font-weight: bold;">Total Energy (E_total)</span
        </div>
    </div>
</div>

<script>
    const canvas = document.getElementById('waveCanvas');
    const ctx = canvas.getContext('2d');

    // UI Elements
    const btnToggle = document.getElementById('modeToggle');
    const freqSlider = document.getElementById('frequency');
    const ampSlider = document.getElementById('amplitude');
    const noiseSlider = document.getElementById('noise');

    const freqVal = document.getElementById('freqVal');
    const ampVal = document.getElementById('ampVal');
    const noiseVal = document.getElementById('noiseVal');

    const elEnergy = document.getElementById('totalEnergy');
    const elEDI = document.getElementById('ediScore');
    const elStatus = document.getElementById('systemStatus');
```

```
let isHackerMode = false;
let time = 0;
let ediHistory = [];

// Mode toggle event
btnToggle.addEventListener('click', () => {
    isHackerMode = !isHackerMode;
    if (isHackerMode) {
        btnToggle.textContent = 'Attacker Mode';
        btnToggle.classList.add('attack');
        ampSlider.disabled = false;
        noiseSlider.disabled = false;
        ampSlider.value = 1.8; // Default asymmetric load during attack
        noiseSlider.value = 0.2; // Default noise during attack
    } else {
        btnToggle.textContent = 'Authorized Mode';
        btnToggle.classList.remove('attack');
        ampSlider.disabled = true;
        noiseSlider.disabled = true;
        ampSlider.value = 1;
        noiseSlider.value = 0;
    }
    updateLabels();
});

// Slider update
function updateLabels() {
    freqVal.textContent = parseFloat(freqSlider.value).toFixed(1);
    ampVal.textContent = parseFloat(ampSlider.value).toFixed(1);
    noiseVal.textContent = parseFloat(noiseSlider.value).toFixed(2);
}

freqSlider.addEventListener('input', updateLabels);
ampSlider.addEventListener('input', updateLabels);
noiseSlider.addEventListener('input', updateLabels);

// Graph drawing and calculation
function draw() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const width = canvas.width;
    const height = canvas.height;
    const centerY = height - 50; // Bottom reference point
    const scaleY = -100; // Y-axis scale (negative draws upwards)

    const f = parseFloat(freqSlider.value) * 0.05;
    const amp1 = parseFloat(ampSlider.value);
```

```
const noiseLvl = parseFloat(noiseSlider.value);

// Draw 1.5 baseline
ctx.beginPath();
ctx.strokeStyle = "rgba(255, 255, 255, 0.2)";
ctx.setLineDash([5, 5]);
ctx.moveTo(0, centerY + (1.5 * scaleY));
ctx.lineTo(width, centerY + (1.5 * scaleY));
ctx.stroke();
ctx.setLineDash([]);

let currentTotalE = 0;
let sumSquareDeviation = 0;

// Phase array and colors
const phases = [
    { offset: 0, color: '#ff9ff3', amp: amp1 }, // Phase 1 (Amplitude modulation
    { offset: (2 * Math.PI) / 3, color: '#feca57', amp: 1 }, // Phase 2
    { offset: (4 * Math.PI) / 3, color: '#48dbfb', amp: 1 }  // Phase 3
];

const points = { p1: [], p2: [], p3: [], sum: [] };

// Calculate data along x-axis (time)
for (let x = 0; x < width; x++) {
    const t = time + x * 0.02;

    // Value of each phase: A * sin^2(f * t + offset)
    let y1 = phases[0].amp * Math.pow(Math.sin(f * t + phases[0].offset), 2);
    let y2 = phases[1].amp * Math.pow(Math.sin(f * t + phases[1].offset), 2);
    let y3 = phases[2].amp * Math.pow(Math.sin(f * t + phases[2].offset), 2);

    // Add noise (Hacker mode)
    if (isHackerMode) {
        y1 += (Math.random() * 2 - 1) * noiseLvl;
        y2 += (Math.random() * 2 - 1) * noiseLvl;
        y3 += (Math.random() * 2 - 1) * noiseLvl;
    }

    const sum = y1 + y2 + y3;

    points.p1.push(y1);
    points.p2.push(y2);
    points.p3.push(y3);
    points.sum.push(sum);

    // Update status with data at the right edge of the screen (current time)
```

```
        if (x === width - 1) {
            currentTotalE = sum;
        }
        // Sum of squared errors for EDI calculation (based on entire window)
        sumSquareDeviation += Math.pow(sum - 1.5, 2);
    }

    // Derive EDI (RMS of deviation)
    const currentEDI = Math.sqrt(sumSquareDeviation / width);

    // Graph drawing function
    function drawPath(data, color, lineWidth) {
        ctx.beginPath();
        ctx.strokeStyle = color;
        ctx.lineWidth = lineWidth;
        for (let x = 0; x < width; x++) {
            const y = centerY + (data[x] * scaleY);
            if (x === 0) ctx.moveTo(x, y);
            else ctx.lineTo(x, y);
        }
        ctx.stroke();
    }

    // Draw individual phases
    drawPath(points.p1, phases[0].color, 1.5);
    drawPath(points.p2, phases[1].color, 1.5);
    drawPath(points.p3, phases[2].color, 1.5);

    // Draw total energy (thick line)
    drawPath(points.sum, '#ff6b6b', 3);

    // Update Status UI
    elEnergy.textContent = currentTotalE.toFixed(3);
    elEDI.textContent = currentEDI.toFixed(3);

    // Determine anomaly based on threshold (0.05)
    if (currentEDI > 0.05) {
        elEnergy.className = 'metric-value danger';
        elEDI.className = 'metric-value danger';
        elStatus.className = 'metric-value danger';
        elStatus.textContent = 'Anomaly Detected!';
    } else {
        elEnergy.className = 'metric-value normal';
        elEDI.className = 'metric-value normal';
        elStatus.className = 'metric-value normal';
        elStatus.textContent = 'Safe';
    }
```

```
        time += 0.05; // Time progression
        requestAnimationFrame(draw);
    }

    // Start animation
    draw();
</script>
```

6. Conclusion

The proposed system enhances security by generating time-varying phase rhythms (TSPM). It provides high-precision real-time detection via EDI and ensures performance in high-speed environments via DPDK/eBPF. It maintains user convenience through automatic synchronization and sliding window correction.

References

1. [Patent] KR102574205B1: Method and apparatus for detecting network attacks
2. [Patent] KR101859032B1: Active cyber defense system using data path variation
3. [Patent] KR102145326B1: Intrusion detection and defense system using intelligent deception technology